To create the client, you need the settings and the client itself. With the protobuf file and Akka gRPC, you get a client when the code is compiled. A `LocatorServiceClient` and the settings are available via the Akka gRPC module `GrpcClientSettings`:

```
import example.locator.grpc.LocatorServiceClient
import akka.grpc.GrpcClientSettings

class LocatorServiceSpec ... {
  ...

  "A Source" should {
    "be able to keep consuming from a failed and restored service" ...

      val clientSettings =                          ❶
        GrpcClientSettings
          .connectToServiceAt("127.0.0.1", 8080)
          .withTls(false)

      val client: LocatorService =                  ❷
        LocatorServiceClient(clientSettings)
  ...
}
```

❶ Settings to connect to the locator service

❷ Creates a client for the locator service

The source is similar for the restart. You need the restart settings and the client. This time, the client calls the `follow` method from the protobuf definition. As you have seen, external calls are wrapped in futures; in this case, the future needs to be wrapped in a `futureSource`:

```
import akka.stream.scaladsl.RestartSource
import akka.stream.RestartSettings
import example.locator.grpc.{IoT, Location}
...

class LocatorServiceSpec ... {
  ...
```